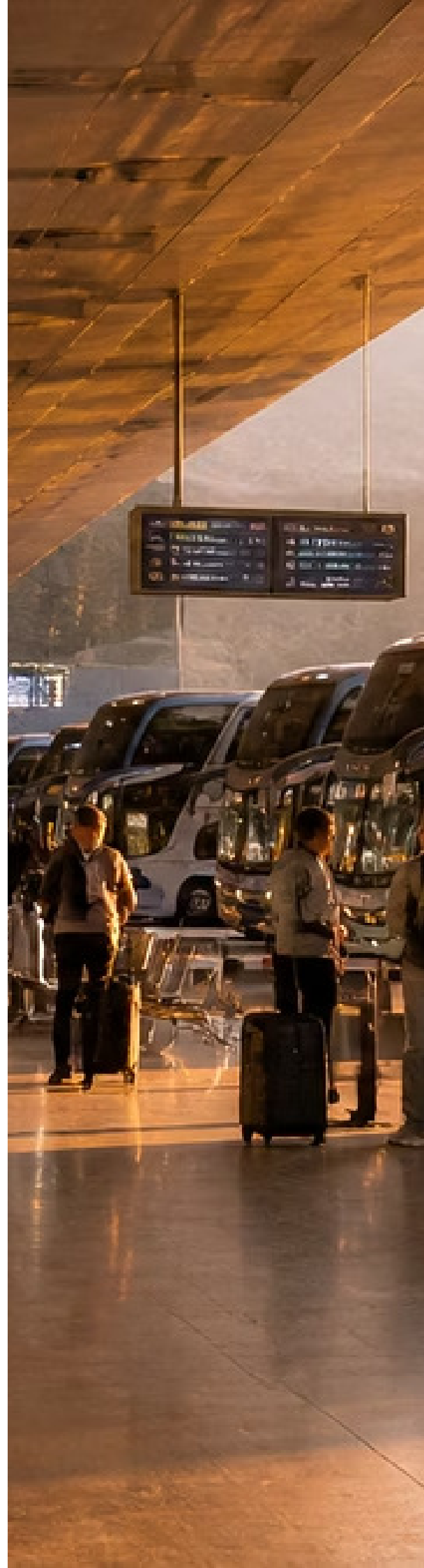


Reto Práctico

Motor de Tarifas para Pasajes Terrestres

¡Felicidades por llegar al final del módulo! Ha llegado el momento de aplicar la teoría a la realidad. Imaginen que han sido contratados como desarrolladores Junior para una popular plataforma chilena de venta de pasajes de bus, similar a Turbus o Kupos.

Su misión es construir el **algoritmo central** en un archivo JavaScript que calcule el precio final de un pasaje interurbano para la ruta **Santiago – Rancagua**. Este algoritmo interactuará con un archivo HTML básico que solo servirá para cargar el script y mostrar los resultados en la consola del navegador.



Objetivos a Evaluar

Con esta actividad demostrarán su dominio sobre los fundamentos esenciales de JavaScript. Cada fase del reto está diseñada para poner a prueba una competencia específica del módulo.

1

Variables en Memoria

Declaración segura de variables utilizando `const` y `let` con buenas prácticas profesionales.

2

Tipos de Datos Primitivos

Manipulación correcta de `String`, `Number`, `Boolean`, `null` y `undefined`.

3

Operadores

Uso correcto de operadores aritméticos, de comparación y comprensión de la precedencia de operaciones.

4

Control de Flujo

Toma de decisiones algorítmicas mediante condicionales `if / else` aplicadas a reglas de negocio reales.

Fase 1: Variables y Tipos de Datos

Deberán crear dos archivos: `index.html` (con la estructura básica y la etiqueta para enlazar su JS) y `calculadoraTarifas.js`. Todo el trabajo lógico ocurrirá en el archivo JS.

Nomenclatura y Hoisting

Todas las variables deben seguir **camelCase** con nombres estrictamente descriptivos. Está **prohibido** usar.

Fase 2: Operadores y Expresiones Matemáticas

→ Jerarquía de Operaciones

Creen una expresión matemática para calcular el precio base más el IVA, aplicando un descuento primero. Es **obligatorio el uso de paréntesis** para demostrar comprensión de la precedencia de operadores.

→ Operador de Incremento

Utilicen un operador de incremento (`++`) para sumar un "cargo por servicio de emisión" de **\$1.000 CLP** al total calculado.

→ Igualdad Estricta y Conversión de Datos

Simulen que la edad del usuario llegó desde el HTML como cadena de texto (ej. `"65"`). Demuestren por qué la igualdad débil (`==`) es peligrosa, conviertan el dato y apliquen **igualdad estricta** (`===`).

Fase 3: Toma de Decisiones con Condicionales

Deberán implementar un bloque condicional `if / else if / else` que altere el precio final según las siguientes **reglas de negocio** de la plataforma:

1

✓ Validación de Destino

Si el destino **NO es exactamente** "Rancagua" (evaluado con `===`), el programa debe detener los cálculos y arrojar un mensaje de error por consola.

2

👴 Descuento Tercera Edad

Si el usuario tiene **65 años o más**, se aplica un **50% de descuento** sobre la tarifa base.

3

🎓 Descuento Estudiante

Si no es tercera edad pero su variable booleana de **pase TNE es verdadera**, se aplica un **20% de descuento**.

4

∅ Gestión de Vacíos

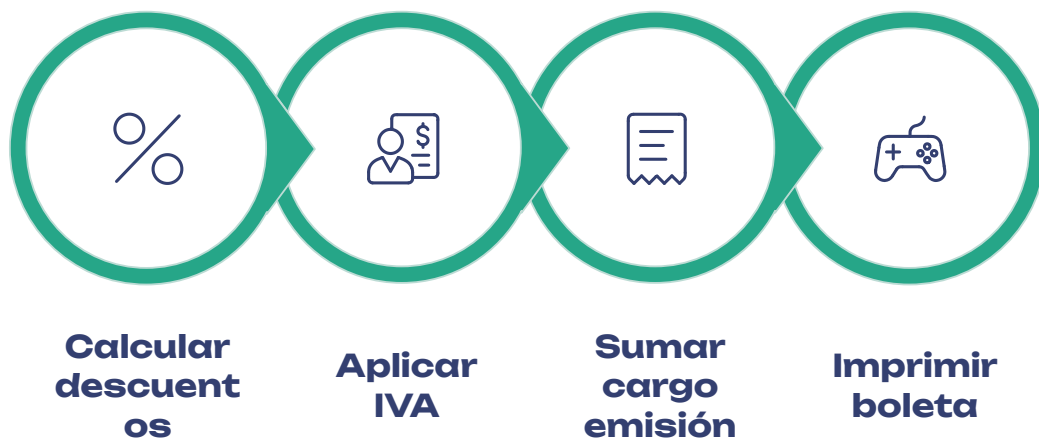
Si la variable del código promocional es estrictamente `null` o `undefined`, **no se aplica ningún descuento extra**.

Fase 4: Boleta Virtual con Template Literals

Una vez que el algoritmo procese todos los descuentos, impuestos y recargos, deben generar una **"Boleta Virtual"** que se imprimirá en la consola mediante `console.log`.

Esta boleta debe construirse utilizando **Template Literals (Backticks)**. Debe ser un `String` multilínea que interpole las variables para mostrar la información completa del pasaje.

- ❗ La boleta debe incluir obligatoriamente: **nombre del pasajero, destino, si se aplicó algún descuento** y el **precio final a pagar** con todos los cargos incluidos.



El uso de Template Literals no es solo una preferencia estética: permite construir strings multilínea de forma legible, evitando la concatenación con `+` y haciendo el código más mantenible y profesional, tal como se hace en la industria.

Entregables

Deberán subir a la plataforma su archivo `.js` debidamente comentado. Recuerden que en programación no solo importa que el algoritmo llegue al resultado correcto: el código debe ser **legible, seguro y seguir los estándares de la industria** aprendidos en este módulo.

Archivo requerido

Subir `calculadoraTarifas.js` con comentarios que expliquen cada decisión técnica: por qué se evitó `var`, el riesgo de la igualdad débil y el comportamiento lexicográfico de los strings.

Calidad del código

Se evaluará la correcta aplicación de `const` y `let`, el uso de paréntesis en expresiones matemáticas, la igualdad estricta en todas las comparaciones y la boleta generada con Template Literals.

Lógica de negocio

El algoritmo debe manejar correctamente los cuatro escenarios: destino inválido, pasajero tercera edad, estudiante con TNE y usuario sin descuentos ni código promocional.



¡Mucho éxito construyendo su primer motor de precios! Este reto integra todas las competencias del módulo en un caso de uso real de la industria tecnológica chilena.